



**The University of Azad Jammu and Kashmir,
Muzaffarabad**

Name Kamal Ali Akmal

Course Name	Data Structure & Algorithm
Submitted to	Engr. Asma Javaid
Semester	3rd
Session	2024-2028
Roll No	2024-SE-38
Lab	Open Ended Lab
Submission date	16 January 2026

Graphs in DSA Using C++

1.Graph

A **graph** is a non-linear data structure consisting of a set of **vertices (nodes)** and a set of **edges** that connect pairs of vertices.

Mathematically:

A graph is represented as: $G = (V, E)$

Where:

- **V** = set of vertices
- **E** = set of edges

2.Basic Graph Terminology

Term	Description
Vertex	A node in a graph
Edge	A connection between two vertices
Degree	Number of edges connected to a vertex
Path	A sequence of connected vertices
Cycle	A path that starts and ends at the same vertex
Connected Graph	Every vertex is reachable

3.Types of Graphs

a) Directed Graph

Edges have a direction ($A \rightarrow B$).

b) Undirected Graph

Edges have no direction ($A — B$).

c) Weighted Graph

Edges have weights (cost, distance, etc.).

4.Graph Representations

a) Adjacency Matrix

A 2D array where rows and columns represent vertices.

b) Adjacency List

An array of linked lists where each vertex stores its adjacent vertices.

5.Graph Traversal Algorithms

- **BFS (Breadth First Search)** – Uses a **queue**.
- **DFS (Depth First Search)** – Uses **recursion or stack**.

Implementation Tasks (C++)

Program 1: Graph Representation Using Adjacency Matrix

```
#include <iostream>
using namespace std;
// Program 1: Graph Representation Using Adjacency Matrix
int main(){
    int vertices;
    cout<<"Enter number of vertices: ";
    cin>>vertices;

    int adjMatrix[10][10]={0};

    int edges;
    cout<<"Enter number of edges: ";
    cin>>edges;

    for(int i=0; i<edges; i++){
        int u,v;
        cout<<"Enter edge (u v): ";
        cin>>u>>v;

        adjMatrix[u][v]=1;
        adjMatrix[v][u]=1;    // Undirected graph
    }

    cout<<"\nAdjacency Matrix:\n";
    for(int i=0; i<vertices; i++) {
        for(int j=0; j<vertices; j++) {
            cout<<adjMatrix[i][j]<<" ";
        }
        cout<<endl;
    }

    return 0;
}
```

Output:

```
Enter number of vertices: 5
Enter number of edges: 6
Enter edge (u v): 0 1
Enter edge (u v): 0 2
Enter edge (u v): 1 2
Enter edge (u v): 1 3
Enter edge (u v): 2 4
Enter edge (u v): 3 4

Adjacency Matrix:
0 1 1 0 0
1 0 1 1 0
1 1 0 0 1
0 1 0 0 1
0 0 1 1 0

-----
Process exited after 362.9 seconds with return value 0
Press any key to continue . . .
```

Program 2: Graph Representation Using Adjacency List

```
#include <iostream>
#include <vector>
using namespace std;
//Program 2: Graph Representation Using Adjacency List
int main(){
    int vertices;
    cout<<"Enter number of vertices: ";
    cin>>vertices;

    vector<int> adjList[10];

    int edges;
    cout<<"Enter number of edges: ";
    cin>>edges;

    for(int i=0; i<edges; i++){
        int u,v;
        cout<<"Enter edge (u v): ";
        cin>>u>>v;

        adjList[u].push_back(v);
        adjList[v].push_back(u); // Undirected graph
    }

    cout << "\nAdjacency List:\n";
    for(int i=0; i<vertices; i++){
        cout<<i<<" ->";
        for(int j : adjList[i]) {
            cout<<j<<" ";
        }
        cout<<endl;
    }
    return 0;
}
```

Output:

```
Enter number of vertices: 5
Enter number of edges: 6
Enter edge (u v): 0 1
Enter edge (u v): 0 2
Enter edge (u v): 1 2
Enter edge (u v): 1 3
Enter edge (u v): 2 4
Enter edge (u v): 3 4

Adjacency List:
0 ->1 2
1 ->0 2 3
2 ->0 1 4
3 ->1 4
4 ->2 3

-----
Process exited after 97.12 seconds with return value 0
Press any key to continue . . .
```

Program 3: BFS Traversal Using Queue

```
#include <iostream>
#include <queue>
using namespace std;
//Program 3: BFS Traversal Using Queue
int adjMatrix[10][10], visited[10];
int vertices;

void BFS(int start){
    queue<int> q;
    visited[start]=1;
    q.push(start);

    while(!q.empty()){
        int v=q.front();
        q.pop();
        cout<<v<<" ";

        for(int i=0; i<vertices; i++){
            if(adjMatrix[v][i]==1 && visited[i] == 0) {
                visited[i]=1;
                q.push(i);
            }
        }
    }
}

int main(){
    cout<<"Enter number of vertices: ";
    cin>>vertices;

    cout<<"Enter adjacency matrix:\n";
    for(int i= 0; i<vertices; i++) {
        for(int j=0; j<vertices; j++) {
            cin>>adjMatrix[i][j];
        }
        visited[i]=0;
    }

    int start;
    cout<<"Enter starting vertex: ";
    cin>>start;

    cout<<"BFS Traversal: ";
    BFS(start);
    return 0;
}
```

Output:

```
Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 1 1 0
1 1 0 0 1
0 1 0 0 1
0 0 1 1 0
Enter starting vertex: 0
BFS Traversal: 0 1 2 3 4
-----
Process exited after 106.6 seconds with return value 0
Press any key to continue . . .
```

Program 4: DFS Traversal Using Recursion

```
#include <iostream>
using namespace std;
// Program 4: DFS Traversal Using Recursion

int adjMatrix[10][10], visited[10];
int vertices;

void DFS(int v){
    visited[v]=1;
    cout<<v<<" ";

    for(int i=0; i<vertices; i++) {
        if(adjMatrix[v][i]==1 && visited[i] == 0) {
            DFS(i);
        }
    }
}

int main(){
    cout<<"Enter number of vertices: ";
    cin>>vertices;

    cout<<"Enter adjacency matrix:\n";
    for(int i=0; i<vertices; i++){
        for(int j=0; j<vertices; j++) {
            cin>>adjMatrix[i][j];
        }
        visited[i]=0;
    }

    int start;
    cout<<"Enter starting vertex: ";
    cin>>start;

    cout<<"DFS Traversal: ";
    DFS(start);
    return 0;
}
```

Output:

```
Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 1 1 0
1 1 0 0 1
0 1 0 0 1
0 0 1 1 0
Enter starting vertex: 0
DFS Traversal: 0 1 2 4 3
-----
Process exited after 108 seconds with return value 0
Press any key to continue . . . |
```